Calculabilitate și Complexitate

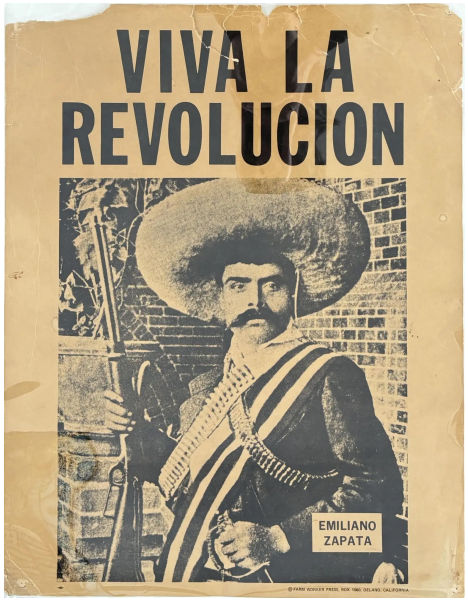
Cursul 1: Programe mici. Surprize mari.

Gabriel Istrate

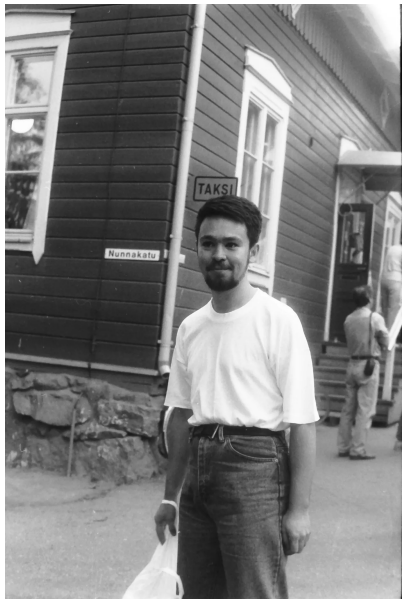
Anul III Semestrul I, 2026–2027

Google Classroom Code: kbl4g576









Surse:

- Poster, Delano, Ca. Farm Workers Press, [ca. 1970].
- Radio Europa Liberă. "Romanian Revolution. Then and Now"
- Poză arhiva proprie. Naantali (Turku), Finlanda, iunie 1993.

21-25 Octombrie. Voi fi la Turku pentru o conferinţă (dupa 33 de ani!)
Orele vor fi realocate

"Revoluţia" voastră

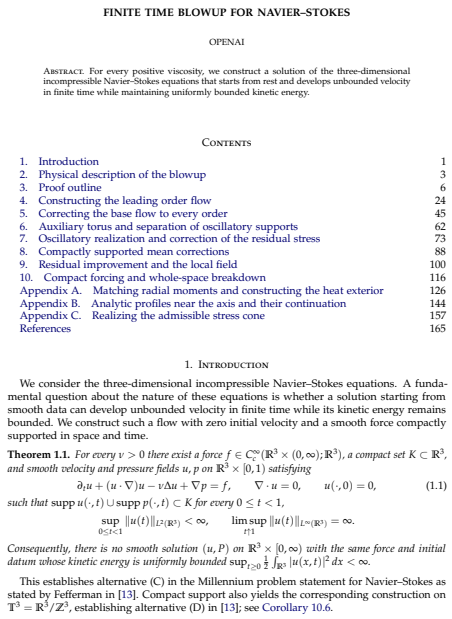


- Posibile surse de nelinişte (pentru voi)

Intrebarea pe care v-o puneţi probabil

La ce bună educaţia universitară in informatică intr-o lume (pe cale de a fi) dominată de A.I.?

Nu sunteţi singurii ....



FINITE TIME BLOWUP FOR NAVIER–STOKES

OPENAI

Abstract. For every positive viscosity, we construct a solution of the three-dimensional incompressible Navier–Stokes equations that starts from rest and develops unbounded velocity in finite time while maintaining uniformly bounded kinetic energy.

Contents

1. Introduction 1
2. Physical description of the blowup 3
3. Proof outline 6
4. Constructing the leading order flow 24
5. Correcting the base flow to every order 45
6. Auxiliary torus and separation of oscillatory supports 62
7. Oscillatory realization and correction of the residual stress 73
8. Compactly supported mean corrections 88
9. Residual improvement and the local field 100
10. Compact forcing and whole-space breakdown 116
Appendix A. Matching radial moments and constructing the heat exterior 126
Appendix B. Analytic profiles near the axis and their continuation 144
Appendix C. Realizing the admissible stress cone 157
References 165

1. Introduction

We consider the three-dimensional incompressible Navier–Stokes equations. A fundamental question about the nature of these equations is whether a solution starting from smooth data can develop unbounded velocity in finite time while its kinetic energy remains bounded. We construct such a flow with zero initial velocity and a smooth force compactly supported in space and time.

Theorem 1.1. *For every $\nu > 0$ there exist a force $f \in C_c^\infty(\mathbb{R}^3 \times (0,\infty);\mathbb{R}^3)$, a compact set $K \subset \mathbb{R}^3$, and smooth velocity and pressure fields u, p on $\mathbb{R}^3 \times [0,1)$ satisfying*

$$\partial_t u + (u\cdot\nabla)u - \nu\Delta u + \nabla p = f, \qquad \nabla\cdot u = 0, \qquad u(\cdot,0) = 0, \tag{1.1}$$

such that $\operatorname{supp} u(\cdot,t) \cup \operatorname{supp} p(\cdot,t) \subset K$ *for every* $0 \le t < 1$,

$$\sup_{0\le t<1} \|u(t)\|_{L^2(\mathbb{R}^3)} < \infty, \qquad \limsup_{t\uparrow 1} \|u(t)\|_{L^\infty(\mathbb{R}^3)} = \infty.$$

Consequently, there is no smooth solution (u,P) on $\mathbb{R}^3 \times [0,\infty)$ with the same force and initial datum whose kinetic energy is uniformly bounded $\sup_{t\ge 0} \frac{1}{2}\int_{\mathbb{R}^3} |u(x,t)|^2\,dx < \infty$.

This establishes alternative (C) in the Millennium problem statement for Navier–Stokes as stated by Fefferman in [13]. Compact support also yields the corresponding construction on $\mathbb{T}^3 = \mathbb{R}^3/\mathbb{Z}^3$, establishing alternative (D) in [13]; see Corollary 10.6.

1

- 8 Septembrie 2026: OpenAI anunţă o demonstraţie pentru problema Navier-Stokes, una din primele 7 probleme deschise ale matematicii.
- 9 Octombrie (astă noapte!): OpenAI a publicat demonstraţii pentru 722 de probleme de matematică.
- **criză de sens in matematică.**

La ce bună ~~educatia universitară in informatică~~ matematica/ştiinţa intr-o lume dominată de AI?

Mit versus Realitate in Ştiinţa din era A.I.

Rezultatele noi/importante obţinute cu AI:

- In majoritate tot de la oameni care se pricep.
- In mâna experţilor: AI o armă super-puternică.
- Lucrările diletanţilor: in majoritate SLOP.

Exemplu de rezultat recent important de informatică teoretică scrise de experţi+ AI: Alman & Williams, 6 Octombrie 2026 (!)

3-SUM: fiind dat un şir de numere $x_1, x_2, \ldots, x_n$, există trei numere $i < j < k$ cu $x_i + x_j + x_k = 0$?

- Algoritm naiv: $O(n^3)$. Algoritm "de Leetcode" (incercaţi !): $O(n^2)$.
- noul algoritm: $O(n^{1.9992})$
- Plus: **All Pairs-Shortest Paths** (Floyd-Warshall, etc.) $O(n^{2.9995})$.

Va fi şi mai greu de "publicat bine" in ştiinţă decât până acum!

(Mulţi dintre) voi şi "teoria"



MY SUBREDDITS HOME - POPULAR - ALL - USERS | EUROPE - MACHINELEARNING - MATH - ANNOUNCEMENTS - EINK - CASUALRO - PRO EDIT »

PROGRAMARE comments DataUnfair (2) | | | preferences | logout

121

Voi trisati la testele de recrutare? (self.programare)
submitted 1 year ago by Aromatic-Chance-8482

Salut, am primit recent niste teste pe mail pentru pre-screening la cateva joburi...

bai baiatule nu m-am simtit niciodata atat de prost, intrebarile pareau incredibil de dificile asa ca automat am folosit AI.

Acum nu stiu daca e o problema daca ai cam la toate raspuns corect si poate erau intrebari capcana si pentru AI, dar efectiv realizez ce nivel jos am la chestii ce tin de programare in sine, nu mai zic de algoritmica si CI, sau poate ce pretentii mari sunt. Mentionez ca momentan am un job pe 15.000 lei si ma simt lejer cu munca, iar jobul asta ar fi oferit doar 16.000.

Dar ma intreb oare sunt doar eu in situatia asta? fortat sa trisez ca sa nu imi ia fața altul care nu stie dar foloseste doar chatGPT?

74 comments share save hide report crosspost

all 74 comments
sorted by: best

save content policy formatting help

[–] mihaicl1981 Kotlin 76 points 1 year ago

search

this post was submitted on 23 Aug 2025
121 points (88% upvoted)
shortlink: https://redd.it/1my8ge

Submit a new link
Submit a new text

programare

leave

Show my flair on this subreddit.
It looks like:
DataUnfair (edit)

created by JetFuelCereals
a community for 6 years

MODERATORS

MESSAGE THE MODS

mikezyisra
:rust_logo::cpp_logo::c_logo::scala_logo::vim_l
cosmin14 :java_logo:
TurtleSoso
ViorelMocanu
AutoModerator
deodorel
Cuza

about moderation team »

De ce nu invăţăm doar "ce ne place/ce avem nevoie la job"?

Ce făceau şcolile de elită cu mult inainte de A.I.


The Freshman Fishwrap / IHTFP Hack Gallery

Fire Hose Drinking Fountain

Location: Building 16, 1st floor lobby
Date: 12/9/91



Former MIT President ['71-'80] Jerome Weisner coined a colorful and often quoted description of the MIT educational experience:

- ``Getting an Education from MIT is like taking a drink from a Fire Hose."

Most students and faculty agree the analogy is quite appropriate.

In 1991, a group of hackers managed to embody this common expression by turning a fire hydrant into a drinking fountain in front of the largest lecture hall on campus (26-100, where many freshman get hosed daily). The fire hydrant drinking fountain made its debut during the final week of classes for the fall term, a classic time when most people are feeling quite drenched

Ce va trebui să stăpâniţi in era A.I.

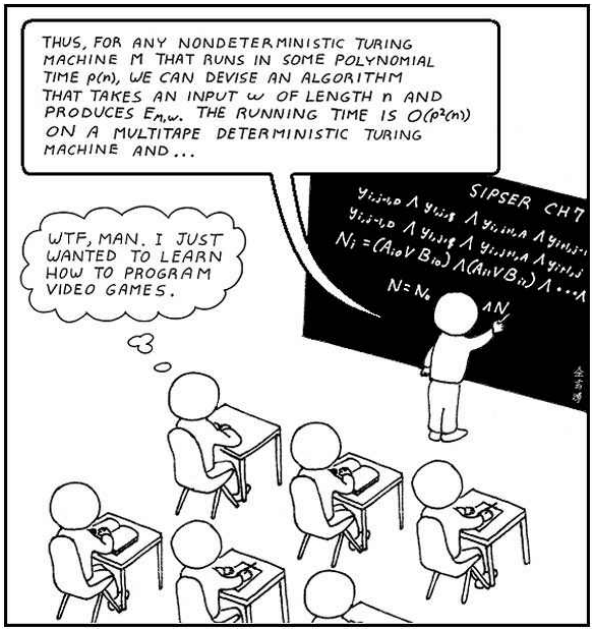


- Poză generată (cu AI) de colegii voştri de la Timişoara.
- Am rezistat tentaţiei sa o "adaptez" la UNIBUC.

- Companiile vor cere mai mult de la voi **pentru că pot.**
- Era AI: informaţia: ieftină, experienţa/discernământul: scump !
- "De ce nu se angajează decât seniori?"
- Senior = om cu experienta/discernământ.

Discernământul se poate educa invăţând lucruri "teoretice"/grele (se testeaza Leetcode NU pentru structuri de date/algoritmi)

Cursul acesta

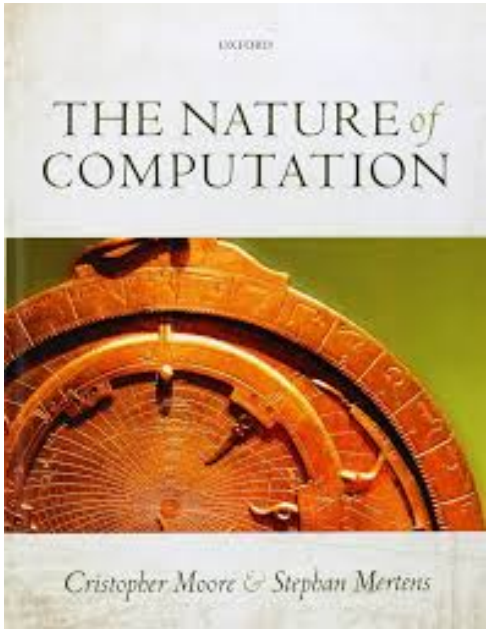


- **NU** vă predă ceva util la job.
- idei importante în informatică, văzută ca o știință
- De obicei foarte matematic.
- Sursa pozei: aici.

Cum il voi preda eu:

- Mai putin matematic. Voi folosi scuza simplificării pentru a vă prezenta unele lucruri mai relevante/actuale, destule nonstandard intr-un astfel de curs.
- Incerc sa-l fac **mai interesant**, nu mai uşor.

Materiale. Reguli.



- **NU** există manual oficial. "Cel mai" manual: Cris Moore & Stefan Mertens "The Nature of Computation".
- Numeroase materiale (handouts, videoclip-uri, programe) pe pagina cursului. Folosiţi-le!

Reguli:

- **NU** fac prezenţa la curs. Prezenţa la seminar (politica universităţii) obligatorie.
- Pot exista quiz-uri (neanunţate) la curs/seminar.
- **SUPER IMPORTANT:** Fără telefoane mobile/laptop-uri/alte device-uri deschise la curs!

Notare (primul examen)

- Pana la nota 6, inclusiv: nivelul 1, test grilă (+quiz-urile).
- 6-9 inclusiv: nivelul 2, probleme. Punctaj minim pentru nivelul 2.
- 10: proiect (pdf/cod)+ videoclip prezentare+(posibil) intrebări.
- Examene ulterioare: model simplificat.

Reguli proiect (altfel nu contează!):

- Orice proiect legat (rezonabil) de tematica cursului. **Voi vă alegeţi tema!** NU prezint o listă. Poate fi continuat intr-o licenţă!
- Tema: una complet nouă, care nu poate fi găsită (in mod rezonabil) in literatură!
- Poate fi un proiect in care nu aţi avut succes (dacă explicaţi care a fost ideea, şi de ce nu a mers)
- Puteţi folosi LLM-uri dar (1). discutaţi asta (2). inţelegeţi TOT ce livraţi (intrebări !) (3). o idee/contribuţie proprie. (4). interesant.

Cum cunoaştem natura

- simţuri/explorare.
- imaginaţie.
- raţionament.
- logicã/matematicã.
- calcul
- (2026-) Inteligenţa artificială?

Leibniz: „Calculemus!”



C. B. Francke, circa 1695,

domeniu public

Biografie: MacTutor. „Calculemus” = „Să calculăm”

Gottfried Wilhelm Leibniz
1646–1716

Matematician, filosof și diplomat german.

- Dezvoltă calculul diferențial și integral, independent de Newton.
- Studiază aritmetica binară și construiește o mașină de calcul.
- Visează la un limbaj în care raționamentele devin calcule.

David Hilbert



Portret: autor necunoscut,

circa 1912, domeniu public

Biografie: MacTutor, University of St Andrews

1862–1943

Matematician german, figură centrală la Göttingen.

- În 1900 propune celebra listă de 23 de probleme.
- Contribuie la geometrie, algebră și analiză.
- Caută fundamente sigure pentru matematică.

Matematica ar putea fi algoritmizată?

Programul lui Hilbert, foarte schematic

Formalizăm matematica și justificăm siguranța sistemului prin metode controlabile.

O întrebare înrudită: problema deciziei

Există o procedură care decide dacă orice formulă a logicii de ordinul întâi este validă?

Notă: Verificarea unei demonstrații e foarte diferită de găsirea unui răspuns/demonstraţii!

Şi totuşi ...

Important

Un model (matematic/computaţional) este o reprezentare a naturii, nu natura.

Programe simple, comportament complex.

Putem înțelege fiecare instrucțiune și totuși să nu înțelegem complet comportamentul programului.

Trei exemple:

- Ackermann.
- Goodstein.
- Collatz.

Funcția Ackermann

```
def A(m, n):
    if m == 0:
        return n + 1
    if n == 0:
        return A(m - 1, 1)
    return A(m - 1, A(m, n - 1))
```

Intrări: $m, n \in \{0, 1, 2, \ldots\}$.

Predicție înainte de rulare

Cât credeți că este $A(3, 4)$? Dar $A(4, 1)$?

Demo: numărăm și apelurile recursive

Varianta Ackermann–Péter. Definiție și istorie: NIST

Numerele cresc repede

$A(m,n)$	$n=0$	$n=1$	$n=2$
$m=0$	1	2	3
$m=1$	2	3	4
$m=2$	3	5	7
$m=3$	5	13	29
$m=4$	13	65 533	$2^{65\,536}-3$

$A(4,2)$ are **19 729 de cifre zecimale.**

$$A(4,3) = 2^{\left(2^{65\,536}\right)} - 3.$$

Chiar și scrierea rezultatului poate deveni problema.

Wilhelm Ackermann



Portret: autor necunoscut,

domeniu public

Biografie: MacTutor, University of St Andrews

1896–1962

Logician german, student al lui Hilbert.

- În 1928 construiește o funcție calculabilă care depășește recursia primitivă.
- Lucrează și ca profesor de liceu.

Funcția este calculabilă și definită pentru orice intrare.

Ackermann în informatică

În structuri de date

Inversa lui Ackermann, $\alpha(n)$, crește extrem de lent. Apare în analiza structurii `Union-Find`.

La frontiera cercetării

2021/2022: unele probleme au, în cazul general, complexitate de tip Ackermann.

2025/2026: un rezultat recent folosește inversa lui Ackermann în adâncimea unor circuite pentru coduri corectoare de erori.

Czerwiński–Orlikowski, FOCS 2021, proceedings 2022

Yuan Li, preprint 2025, revizuit în 2026

Regula lui Goodstein

Pornim de la un număr natural și de la baza 2.

1. Scriem numărul în baza curentă, **inclusiv exponenții, recursiv**.
2. Mărim baza cu 1, peste tot în expresie.
3. Scădem 1. Repetăm până la 0.

Pare o strategie bună pentru a ajunge la zero?

Mărim baza, apoi scădem doar 1.

R. L. Goodstein, On the Restricted Ordinal Theorem, 1944

„Ereditar” înseamnă peste tot

Pentru valoarea inițială 4:

$$4 = 2^2 = 2^{(2^1)}.$$

Schimbăm **toate** aparițiile bazei 2 în 3:

$$3^{(3^1)} - 1 = 27 - 1 = \boxed{26}.$$

Dacă am lăsa exponentul 2 neschimbat, am obține $3^2 - 1 = 8$.

Aceasta ar fi **o altă regulă**.

Două începuturi foarte diferite

Baza	2	3	4	5	6	7
De la 3	3	3	3	2	1	0
De la 4	4	26	41	60	83	109

Același cod. O diferență de 1 la intrare.

Vot rapid

Șirul de la 4 va ajunge vreodată la zero?

Demo: schimbăm valoarea inițială

Un nucleu Python pentru Goodstein

```
def bump(n, b):
    if n < b:
        return n
    total, exponent = 0, 0
    while n:
        n, digit = divmod(n, b)
        if digit:
            total += digit * (b + 1)**bump(exponent, b)
        exponent += 1
    return total

# Un pas, daca n > 0:
n = bump(n, b) - 1
b += 1
```

`bump` schimbă baza și în exponenți.

Goodstein: ce știm

1944: teorema lui Goodstein

Orice șir Goodstein standard ajunge la zero.

1982: Kirby și Paris

Aritmetica Peano nu poate demonstra afirmația pentru **toate** valorile inițiale.

Putem demonstra teorema într-un sistem mai puternic.

Putem calcula fiecare șir. Timpul necesar poate fi enorm.

Kirby–Paris, Accessible Independence Results for Peano Arithmetic, 1982

Şirul lui Collatz

```
def collatz(n):
    while n != 1:
        n = n // 2 if n % 2 == 0 else 3*n + 1
    return 1
```

Pornim de la un întreg pozitiv.

$$6,\ 3,\ 10,\ 5,\ 16,\ 8,\ 4,\ 2,\ 1.$$

Întrebarea

Se termină pentru **orice** intrare?

Demo: încercăm 27

27 nu se grăbește

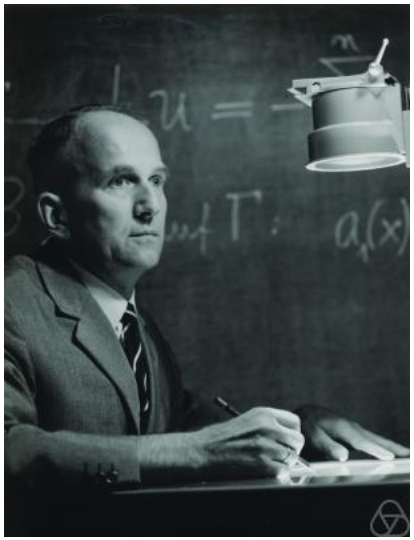
$$27,\ 82,\ 41,\ 124,\ 62,\ 31, \ldots$$

Pași până la prima atingere a lui 1	**111**
Cea mai mare valoare întâlnită	**9 232**

Un comportament accidentat poate apărea într-un program determinist.

Demo: orbita și graficul ei

Collatz: o problemă deschisă



Lothar Collatz
1910–1990

Konrad Jacobs, CC BY-SA 2.0 DE

1937: conjectura asociată lui Collatz.

2019/2022: Tao demonstrează un rezultat pentru „aproape toate” intrările, într-un sens precis.

2025: Bařina verifică toate intrările pozitive sub 2^{71}.

Octombrie 2026: conjectura generală rămâne deschisă.

Tao, Almost all orbits of the Collatz map attain almost bounded values

Verificarea lui Bařina și progresul curent

Trei lucruri diferite

Ackermann	Se termină, dar valorile cresc foarte repede.
Goodstein	Se termină mereu, dar PA nu demonstrează universal acest lucru.
Collatz	Putem calcula pașii. Nu știm dacă toate orbitele ajung la 1.

Ideea de ținut minte

Un experiment oprit la timeout nu dovedește că algoritmul nu se opreşte.

Calculăm ca să prezicem

- Va ajunge programul în această stare?
- Ce se întâmplă cu sistemul dacă schimbăm intrarea?
- Ce acțiune are șanse mai bune să funcționeze?

Patru niveluri ale întrebării

Putem formula problema? Există un algoritm?

Îl putem rula la timp? Ce justifică folosirea rezultatului?

Este calculabilă natura?

Întrebări pe care trebuie să le separăm

Putem simula un model?

Putem prezice un fenomen la precizia dorită?

Putem descrie exact întreaga natură printr-un algoritm?

Sensibilitatea la condițiile inițiale poate face predicția dificilă.

Ideea de ținut minte

O simulare reușită nu demonstrează că toate fenomenele naturale sunt calculabile.

Este calculabil un stol?



Clip offline: 25 de secunde Fișierul video

Karen Roe, Norfolk, 2017. CC BY 2.0. Extras fără sunet

Boids: câteva reguli locale

Craig Reynolds propune modelul în **1987**.

1. **Separare:** păstrează distanța față de vecinii prea apropiați.
2. **Aliniere:** ajustează direcția după vecini.
3. **Coeziune:** apropie-te de grupul din jur.

Mișcarea colectivă apare din interacțiuni locale.

Demo: oprim o regulă și observăm efectul

Reynolds, Flocks, Herds, and Schools, SIGGRAPH 1987

O lume cu două reguli

Game of Life, John Conway, 1970.

- O celulă vie supraviețuiește cu 2 sau 3 vecini vii.
- O celulă moartă devine vie cu exact 3 vecini vii.

Toate celulele se actualizează simultan.

Ce vrem să observăm

Un model foarte simplu poate produce structuri care persistă și se deplasează.

Demo: un glider și câteva celule adăugate de voi

Este un model matematic. Nu este o demonstrație că viața biologică se reduce la aceste reguli.

John von Neumann



Foto: wikispaces, CC BY-SA 3.0
Biografie: Institute for Advanced Study

1903–1957

Născut la Budapesta. Lucrează la Princeton.

- Contribuie la mecanica cuantică și teoria jocurilor.
- Are un rol central în dezvoltarea calculatoarelor cu program memorat.
- Studiază automate care se pot reproduce.

Cum se poate auto-copia o mașină?

O mașină are și **o descriere a modului de a construi mașina**.

Descriere ca instrucțiuni	Constructorul o interpretează și construiește o copie.
Descriere ca date	Copiatorul duplică descrierea pentru noua mașină.

Copia primește și **capacitatea de a repeta procesul**.

von Neumann–Burks, Theory of Self-Reproducing Automata, 1966

Un program Python care se auto-tipărește

Un quine: produce ca ieșire propriul cod sursă.

```
s = 's = %r; print(s %% s)'; print(s % s)
```

- `s` conține un șablon al programului.
- `%r` inserează reprezentarea șirului, cu ghilimele.
- `%%` produce un caracter `%` literal.
- `s % s` completează șablonul cu propriul său conținut.

Nu citește fișierul în care se află.

Este inteligența algoritmică?

Ce putem observa

Programele îndeplinesc sarcini pe care le asociem cu inteligența.

Ce rămâne de discutat

Asta arată că toate formele de inteligență sunt algoritmice?

Ce înseamnă să „înțelegi” și cum ai testa acest lucru?

O întrebare filosofică are nevoie de termeni și criterii, la fel ca o problemă de calcul.

Ar putea IA rezolva orice?

O presupunere foarte optimistă

Antrenăm un model mai mare și îi dăm mai mult timp.

IA pe un calculator obișnuit este tot un calcul executat de un program.

Ideea de ținut minte

Dacă o problemă nu admite niciun algoritm general exact, nici o IA implementată algoritmic nu poate oferi unul.

Poate totuși ajuta pe instanțe, prin aproximări sau prin descoperirea unor idei.

Cursul acesta

Calculabilitate	Există un algoritm care dă răspunsul corect și se oprește pentru orice intrare?
Complexitate	Cât timp, câtă memorie și ce alte resurse îi trebuie?

La început: ce se poate calcula **în principiu**, cu resurse oricât de mari de timp/memorie.

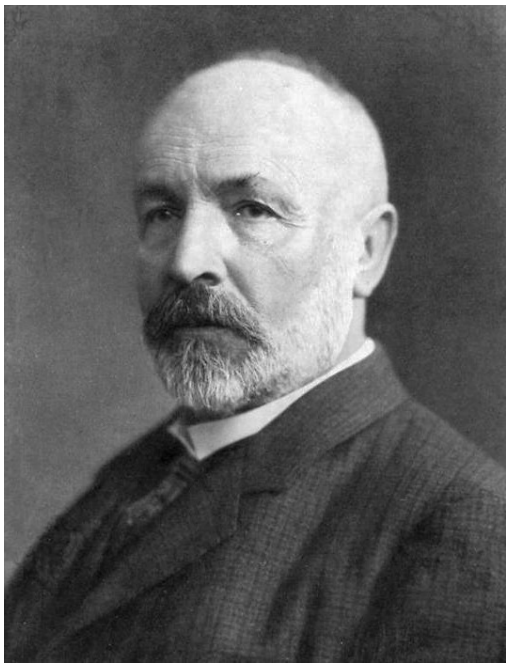
În partea a doua: ce se poate calcula **cu resurse rezonabile**.

Urmează: un rezultat matematic

- Există funcţii $f : \mathbb{N} \to \mathbb{N}$ pe care **nu** le putem calcula ...
- pentru că "sunt mai multe funcţii decât programe care să le calculeze".

Rezultatul **nu** dă un exemplu de funţie pe care nu o putem calcula.
Vom vedea exemple ulterior.

Georg Cantor



Portret: autor necunoscut,

circa 1900, domeniu public
Biografie și cronologie: MacTutor

1845–1918

Născut la Sankt Petersburg, lucrează în Germania.

- Fondator al teoriei mulțimilor.
- Arată că există infinituri de mărimi diferite.
- 1874: nenumărabilitatea numerelor reale.
- 1891: argumentul diagonal.

Cum comparăm două infinituri?

Două mulțimi au aceeași mărime dacă putem face o **corespondență unu-la-unu** între toate elementele lor.

Naturale	0	1	2	3	4	$\cdots$
Pare	0	2	4	6	8	$\cdots$

$$n \longmapsto 2n.$$

Fiecare natural are un partener. Fiecare număr par apare exact o dată.

Programele încap într-o listă

Fixăm un alfabet finit și un limbaj de programare.

1. Enumerăm textele de lungime 0, apoi 1, apoi 2, etc.
2. La fiecare lungime folosim ordinea lexicografică.
3. Păstrăm textele care sunt programe valide.

$$P_0,\ P_1,\ P_2,\ P_3, \ldots$$

Ideea de ținut minte

Există o infinitate numărabilă de programe finite.

O funcție poate cere o listă infinită

O funcție $f : \mathbb{N} \to \mathbb{N}$ asociază fiecărei intrări un rezultat.

$$f(0),\ f(1),\ f(2),\ f(3), \ldots$$

În această definiție nu cerem încă existența unui program.

Întrebarea lui Cantor, în forma care ne trebuie

Putem pune **toate** aceste funcții într-o singură listă?

O listă poate fi infinită. Fiecare poziție are totuși un indice natural.

Lista pe care încercăm să o completăm

Să presupunem că am listat toate funcțiile: $f_0, f_1, f_2, \ldots$.

	0	1	2	3	$\cdots$
f_0	**0**	2	1	4	$\cdots$
f_1	1	**3**	0	1	$\cdots$
f_2	4	0	**2**	3	$\cdots$
f_3	2	1	0	**0**	$\cdots$
$\vdots$					

Citim diagonala: $0, 3, 2, 0, \ldots$.

Demo: un tabel pe care îl puteți modifica

O funcție care scapă listei

Modificăm fiecare element de pe diagonală:

$$\boxed{g(i) = f_i(i) + 1.}$$

În exemplu: $g = 1, 4, 3, 1, \ldots$.

- g diferă de f_0 la intrarea 0.
- g diferă de f_1 la intrarea 1.
- În general, g diferă de f_i la intrarea i.

g nu este niciun rând din listă. Contradicție.

Nici numerele reale nu încap în listă

Presupunem o listă a numerelor din $(0, 1)$:

$$\begin{aligned} r_1 &= 0.1\,0\,1\,1\cdots \\ r_2 &= 0.2\,2\,0\,1\cdots \\ r_3 &= 0.0\,1\,3\,2\cdots \\ r_4 &= 0.1\,2\,1\,1\cdots \\ &\vdots \end{aligned}$$

Cifra i a noului număr este **2** dacă cifra diagonală este 1 și **1** altfel.
Noul număr diferă de fiecare r_i la cifra i.

Funcțiile și numerele reale au aceeași mărime

$$\#\{f : \mathbb{N} \to \mathbb{N}\} = \#\mathbb{R} > \#\mathbb{N}.$$

- O funcție este un șir infinit de numere naturale.
- Putem codifica un asemenea șir într-un șir infinit de biți.
- Șirurile infinite de biți și numerele reale au aceeași cardinalitate.

Aici comparăm mărimi de mulțimi. Nu afirmăm că putem calcula toate cifrele oricărui real.

Există funcții necalculabile

Programe finite	**numărabil de multe**
Funcții $\mathbb{N} \to \mathbb{N}$	**nenumărabil de multe**

Fiecare program calculează cel mult o funcție totală.

Ideea de ținut minte

Sunt mai multe funcții decât programe. Unele funcții nu pot fi calculate de niciun program.

Argumentul arată **existența**. Vom căuta și probleme explicite.

Patru idei cu care plecăm

1. Codul simplu poate avea comportament greu de anticipat.
2. Experimentele, demonstrațiile și predicțiile au garanții diferite.
3. Calculabilitatea privește existența unui algoritm general.
4. Numărul programelor este prea mic pentru toate funcțiile.

Data viitoare

Cum definim precis „algoritm”? Cum arată o problemă explicită pe care niciun algoritm nu o poate decide?

Intrebări de final.

1. O buclă a rulat un minut fără rezultat. Ce știm de aici?
2. Ce anume modificăm în argumentul diagonal?
3. Cum poate fi adevărată o afirmație pe care un sistem de axiome nu o demonstrează?

Răspunsuri scurte, în cuvintele voastre.

Anexă: ce spune exact Tao?

Fie $F(N)$ o funcție care tinde la infinit, oricât de lent. Pentru „aproape toate” valorile inițiale N, orbita Collatz ajunge sub $F(N)$. „Aproape toate” înseamnă aici **densitate logaritmică 1**.

Limita rezultatului

Nu afirmă că orbita ajunge la 1.
Nu exclude toate intrările excepționale.

Tao, Forum of Mathematics, Pi 10 (2022), e12. Preprint din 2019

Anexă: ideea dovezii Goodstein

Numerele naturale din șir pot crește. Înlocuim simbolul bazei

printr-un simbol ordinal ω.

- Schimbarea bazei păstrează acest cod ordinal.
- Scăderea lui 1 face codul să scadă strict.
- Nu există un șir infinit strict descrescător de ordinale.

Ideea de ținut minte

Căutăm o mărime care scade, chiar dacă valorile afișate cresc.

Anexă: quine-ul Python

```
s = 's = %r; print(s %% s)'; print(s % s)
```

- `%r` introduce reprezentarea citibilă a șirului.
- `%%` produce un caracter `%` literal.
- Șablonul și șirul inserat refac exact sursa.

```
python python/quine.py
```

Copia textuală nu este încă un proces care execută o nouă copie.

Anexă: funcții codificate în biți

Pentru $f : \mathbb{N} \to \mathbb{N}$, scriem:

$$\underbrace{1 \cdots 1}_{f(0)} 0 \; \underbrace{1 \cdots 1}_{f(1)} 0 \; \underbrace{1 \cdots 1}_{f(2)} 0 \; \cdots$$

Exemplu: $f = 2, 0, 3, \ldots$ produce $110\,0\,1110 \cdots$. Separatorul 0 ne permite să recuperăm toate valorile. Și invers, orice șir de biți este o funcție $\mathbb{N} \to \mathbb{N}$ care ia doar valorile 0 și 1.

Anexă: biți și numere reale

Pentru biții $b_0, b_1, b_2, \ldots$, folosim un real cu cifre ternare 0 sau 2:

$$x = \sum_{i=0}^{\infty} \frac{2b_i}{3^{i+1}}.$$

Șiruri diferite dau numere diferite. În sens invers, codificăm un real prin semn, partea întreagă și o scriere zecimală canonică. **Două injecții dau o bijecție**
prin teorema Cantor–Schröder–Bernstein.

Anexă: pornirea demonstrațiilor

După extragerea arhivei, în folderul cursului:

```
python demo_curs1.py
```

Se deschide interfața în browser. Linkurile „Demo” din PDF duc la aceeași interfață.

Pentru exemple individuale

```
python python/ackermann.py 3 4
python python/goodstein.py 4 -steps 15
python python/collatz.py 27
```

Python 3.10 sau mai nou. Fără biblioteci externe. Video-ul și sursele sunt și atașamente ale PDF-ului.

Surse pentru cele trei programe

- Czerwiński–Orlikowski: Reachability in Vector Addition Systems is Ackermann-complete, 2021/2022.
- Yuan Li: Capacity-Achieving Codes with Inverse-Ackermann-Depth Encoders, 2025/2026.
- Goodstein: On the Restricted Ordinal Theorem, 1944.
- Kirby–Paris: Accessible Independence Results for Peano Arithmetic, 1982.
- Fernández-Duque–Weiermann: A Walk with Goodstein and Ackermann, 2024.
- Wilken: Uniform generalization of Goodstein's theorem and Cichon's independence proof, preprint, 2026.
- Tao: Almost all orbits of the Collatz map attain almost bounded values, 2022.
- Bařina: Improved verification limit for the convergence of the Collatz conjecture, 2025.

Lecturi, imagini și video

- Cristopher Moore și Stephan Mertens, The Nature of Computation, Oxford University Press, 2011.
- Craig Reynolds: Flocks, Herds, and Schools, 1987.
- APPR: documentația Public Safety Assessment.
- Google DeepMind: AlphaEvolve, 2025.
- MacTutor: biografiile lui Ackermann, Leibniz, Hilbert și Cantor.
- Institute for Advanced Study: John von Neumann.
- Portrete: Wikimedia Commons, cu sursa pe slide-ul fiecăruia.
- Clip: Karen Roe, Flock of starlings, CC BY 2.0.
- Bucle Langton: Thomas Schoch, CC BY-SA 3.0.

Mai departe: Conferința lui Tao despre Collatz la IAS.